

Project Report: Console-Based Expense and Income Tracker

Course: Python Essentials

Student Name: Utkarsh Singh

Registration Number: 26BAI10948

Repository URL: <https://github.com/utkarshcodeprofile-blip/Expense-Tracker->

Submission Date: September 30, 2026

1. Introduction & Motivation

Keeping tabs on personal daily finances is an essential habit, yet students often abandon digital record-keeping because existing commercial apps demand account logins, display intrusive advertisements, or require manual internet sync. On the other hand, managing custom spreadsheet sheets via phone or laptop can feel clunky for rapid, on-the-spot entries.

The Expense and Income Tracker was developed to provide a lightweight, local-first alternative designed specifically for terminal environments. By relying exclusively on Python's native libraries, the utility lets users quickly record transactions, verify balances, and manage data without any network requirements, third-party frameworks, or external overhead.

2. Project Objectives

The primary objectives of the application are:

Zero External Dependencies: Built solely on Python 3 and standard library modules (sqlite3), eliminating the need for pip install setups or complex environment isolation.

Persistent Relational Storage: Store transaction records in a structured local SQLite database (transactions.db) to ensure information remains intact between terminal sessions.

Automated Balance Calculation: Dynamically compute and display total expenses, total earnings, and net cash flow directly on user request.

Command-Line Usability: Provide a numbered menu loop designed for command-line execution, equipped with explicit confirmation safeguards against accidental data erasure.

3. System Architecture & Workflow

The program operates using a procedural CLI event loop directly interfaced with an embedded SQLite engine.

```
+-----+
| CLI Main Menu Loop |
+-----+-----+
|
+-----+-----+-----+-----+
|||
vvvv
[1] Add Expense [2] Add Income [3] View Records [4] Clear Data
|||
+-----+-----+ ||
|||
```

v v v

```
INSERT INTO table SELECT * & SUM() DELETE FROM table
```

```
\|/
```

```
\|/
```

```
+-----+-----+
```

```
|
```

v

transactions.db (SQLite3)

Operational Steps

Database Initialization: On script launch, the application opens a connection to transactions.db. If the database file or the transactions table does not already exist, it runs CREATE TABLE IF NOT EXISTS to configure the schema.

Interactive Menu Loop: A continuous while True loop prompts the user to select options numbered 1 through 5.

Immediate Disk Commit: Every write operation calls conn.commit() immediately, preventing unsaved records in the event of an abrupt terminal exit.

4. Database Schema Design

The SQLite database uses a single, normalized table named transactions:

5. Functional Implementation Details

5.1 Transaction Recording (add_transaction)

Accepts a string parameter indicating "expense" or "income". The function prompts for a descriptive string and a numerical amount, writes the record using parameterized SQL placeholders (?) to mitigate injection risks, and immediately commits the transaction.

5.2 Dynamic Aggregation (calculate_total_expenses & calculate_total_income)

Rather than iterating over database records in Python memory, calculations are offloaded to SQL aggregation:

SQL

```
SELECT SUM(amount) FROM transactions WHERE type = 'expense';
```

If no rows match the filter, cursor.fetchone()[0] yields None. A concise fallback expression (total_expenses if total_expenses else 0) guarantees safe numerical operations without triggering type errors.

5.3 Ledger Display & Balance Overview (display_transactions)

Queries all rows using SELECT id, description, amount, type FROM transactions and formats them cleanly for terminal viewing. It then calls the aggregation functions to compute the net balance:

5.4 Data Clearing Safeguard (delete_history)

Before wiping stored entries via DELETE FROM transactions, the program requires explicit user confirmation (y/n). This guard prevents accidental data erasure caused by an unintended keystroke.

6. Execution Walkthrough & Sample Output

Plaintext

Expense and Income Tracker

1. Add Expense

2. Add Income
3. Display Transactions
4. Delete Transaction History
5. Exit

Enter your choice: 2

Enter transaction description: Project Stipend

Enter transaction amount: 2000

Transaction added successfully!

Expense and Income Tracker

1. Add Expense
2. Add Income
3. Display Transactions
4. Delete Transaction History
5. Exit

Enter your choice: 1

Enter transaction description: Reference Books

Enter transaction amount: 350.50

Transaction added successfully!

Expense and Income Tracker

1. Add Expense
2. Add Income
3. Display Transactions
4. Delete Transaction History
5. Exit

Enter your choice: 3

Transactions:

1 - Project Stipend: \$2000.0 (income)

2 - Reference Books: \$350.5 (expense)

Total expenses: 350.5

Total income: 2000.0

Net balance: 1649.5

7. Observations & Practical Considerations

During testing and code evaluation, several behavioral points and potential future refinements were noted:

Numeric Input Handling: The input prompt `float(input(...))` expects clean numerical characters. Non-numeric input raises an unhandled `ValueError`. Encapsulating inputs inside a `try-except` block would make the CLI more resilient against accidental user errors.

Database Lifecycle Management: The connection `conn.close()` is placed outside the main menu loop. Wrapping database operations in context managers or `try-finally` blocks represents an industry-standard practice for resource safety.

Currency Display Formatting: Formatting float amounts using specifiers like `f"${amount:.2f}"` would ensure two decimal places across all currency outputs.

8. Conclusion

The Expense and Income Tracker fulfills the technical specifications of a command-line Python project. It requires zero third-party installation, utilizes SQLite for dependable local persistence, and provides an accessible, distraction-free tool for personal financial management.

9. References

Python Software Foundation: `sqlite3` Standard Library Documentation (<https://docs.python.org/3/library/sqlite3.html>)

SQLite Documentation: Core Syntax, Data Types, and Aggregate Functions (<https://www.sqlite.org/lang.html>)

Column Name	Data Type	Constraints	Purpose
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for each entry
description	TEXT	Default	Description/label of the transaction
amount	REAL	Default	Monetary value stored as a floating-point number
type	TEXT	Default	Categorization tag ("expense" or "income")